

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models 1
Student Name	Saravana Senthilkumar
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers, and to analyze the effect of hyperparameter tuning on classification performance.

2 Problem Statement

Given 57 numerical features extracted from email content, the task is to build a binary classifier that predicts whether an email is spam or ham. Logistic Regression and SVM (with Linear, Polynomial, RBF, and Sigmoid kernels) are trained, tuned via Grid Search / Randomized Search, and compared using accuracy, precision, recall, F1-score, and ROC-AUC.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601
Number of Features	57 numerical features
Number of Classes	2 (Spam, Ham)
Missing Values	None
Train-Test Split	80% train / 20% test (stratified)

4 Exploratory Data Analysis

- Dataset overview: 4601 emails, 57 word/character-frequency and capital-run-length features.
- Statistical summary: features are heavily right-skewed and zero-inflated, typical of sparse text-frequency data.
- Missing value analysis: no missing values found.
- Class distribution: moderately imbalanced, with more ham than spam emails.
- Correlation matrix: individual word-frequency features show weak pairwise correlation but jointly separate the classes.

- Feature distribution: most features concentrated near zero with long right tails.

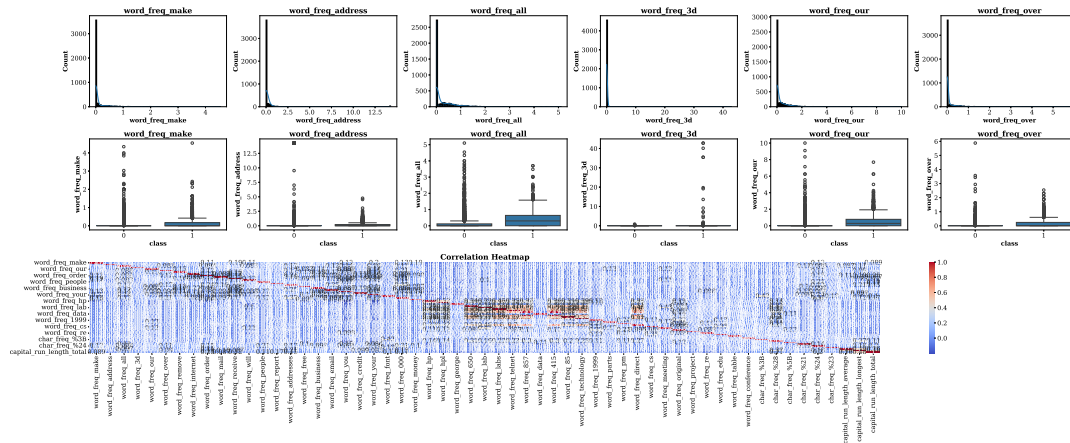


Figure 1: Exploratory Data Analysis of the Spambase dataset.

Inference: The EDA confirms Spambase features are sparse, skewed count-like frequencies rather than normally distributed measurements. This motivates feature standardization before training distance/margin-based models such as SVM, since unscaled skewed features can distort the margin computation. The absence of missing values simplifies preprocessing to primarily scaling.

5 Data Preprocessing

- **Missing value handling:** Not required – no missing values present.
- **Encoding:** Not required – all features and the target are numeric.
- **Feature scaling:** `StandardScaler` applied to standardize all 57 features, critical for both Logistic Regression convergence and SVM margin computation.
- **Train-test split:** 80–20 stratified split to preserve class balance in both sets.
- **Feature engineering:** Not applied; original 57 features used directly.

6 Machine Learning Algorithm

Logistic Regression

Working principle: Models class probability via the sigmoid function:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}} \quad (1)$$

trained by minimizing log-loss. L1 regularization drives weak coefficients to zero (feature selection); L2 shrinks weights smoothly. **Advantages:** Fast, interpretable, provides probabilistic outputs. **Limitations:** Assumes a linear decision boundary in feature space; can underfit non-linear patterns.

Support Vector Machine (SVM)

Working principle: Finds the hyperplane that maximizes the margin between classes; only support vectors influence the boundary. Kernels (Linear, Polynomial, RBF, Sigmoid) implicitly map data into higher-dimensional spaces to capture non-linear separability.

Advantages: Effective in high-dimensional spaces; RBF kernel can model complex non-linear boundaries. **Limitations:** Training can be slow on large datasets; performance is sensitive to kernel choice and hyperparameters (C , γ).

7 Experimental Setup

Python Version	3.10 / 3.14 (venv)
Scikit-Learn Version	Recent stable (<code>scikit-learn</code>)
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Standard CPU (no GPU acceleration used)

8 Hyperparameter Selection

Parameter	Value
Logistic Regression: penalty	L1, L2
Logistic Regression: C	0.01, 0.1, 1, 10, 100
Logistic Regression: solver	liblinear, saga
SVM: kernel	Linear, Polynomial, RBF, Sigmoid
SVM: C	0.1, 1, 10, 100
SVM: γ	scale, auto
SVM: degree (Polynomial)	2, 3, 4
Best Logistic Regression (Grid Search)	$C = 100$, penalty=L1, solver=liblinear (CV acc. 0.9261)
Best SVM (Grid Search)	$C = 10$, γ =scale, kernel=RBF (CV acc. 0.9356)
Best SVM (Randomized Search)	$C \approx 4.77$, degree=4, γ =auto, kernel=RBF (CV acc. 0.9359)

9 Results

Metric	Value (Tuned Logistic Regression)
Accuracy	0.9262
Precision	0.9260
Recall	0.9262
F1-score	0.9259
Training Time (s)	0.2813

Table 2: SVM kernel-wise performance (default hyperparameters)

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9294	0.9293	1.0003
Polynomial	0.7796	0.7568	0.9336
RBF	0.9273	0.9269	0.6188
Sigmoid	0.8849	0.8847	0.5515

Table 3: Final tuned model comparison on the test set

Metric	Logistic Regression	SVM (RBF, tuned)
Accuracy	0.9262	0.9207
Precision	0.9260	0.9206
Recall	0.9262	0.9207
F1 Score	0.9259	0.9205

Table 4: 5-Fold cross-validation accuracy

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Logistic Regression	0.9416	0.9253	0.9348	0.9117	0.9171	0.9261
SVM (RBF)	0.9443	0.9416	0.9348	0.9307	0.9266	0.9356

10 Result Visualization

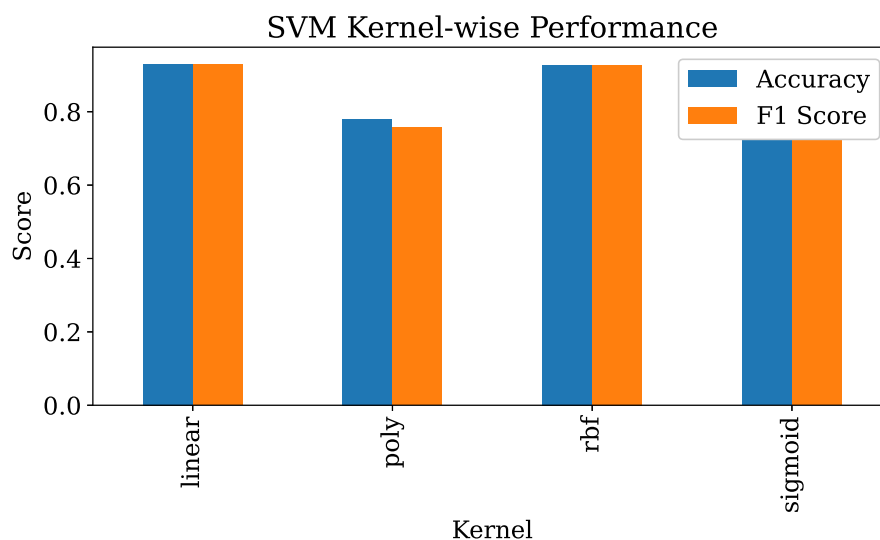


Figure 2: SVM kernel-wise Accuracy and F1 Score comparison.

Inference: The Linear and RBF kernels are the clear top performers, while the Polynomial kernel underperforms sharply with default settings, showing SVM’s strong sensitivity

to kernel choice and hyperparameters. The Sigmoid kernel sits between these extremes, behaving similarly to a single-layer neural activation but without matching the flexibility of RBF. This motivates using RBF as the primary tuning target, which is indeed what the Grid/Randomized Search selected.

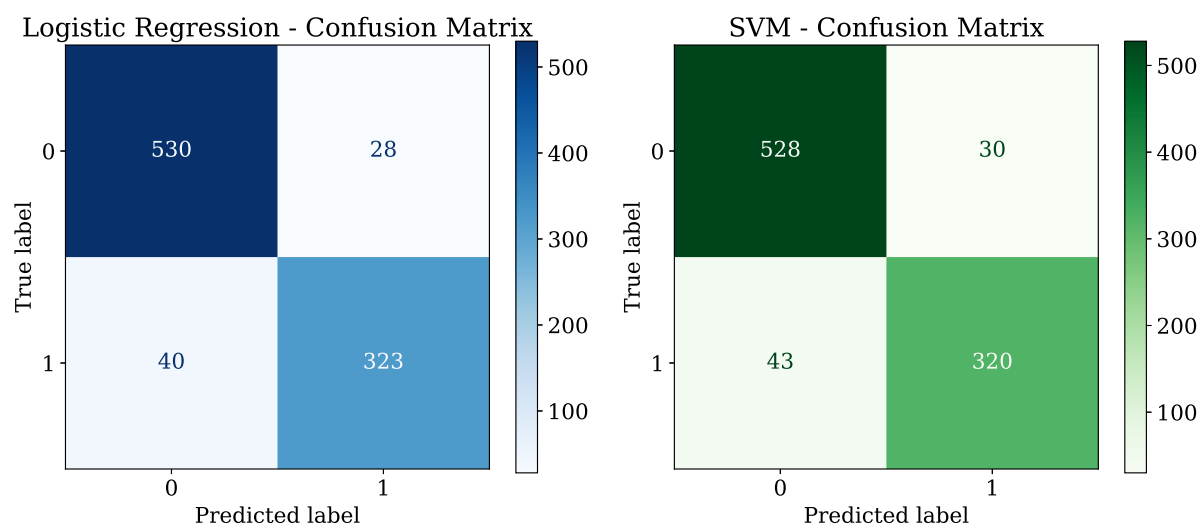


Figure 3: Confusion matrices for Logistic Regression (left) and SVM (right).

Inference: Both models show a similar error pattern: slightly more false negatives (spam predicted as ham) than false positives, which is common in spam classifiers tuned for balanced accuracy rather than recall. The two confusion matrices are very close in magnitude, consistent with the near-identical accuracy scores of both tuned models on the test set.

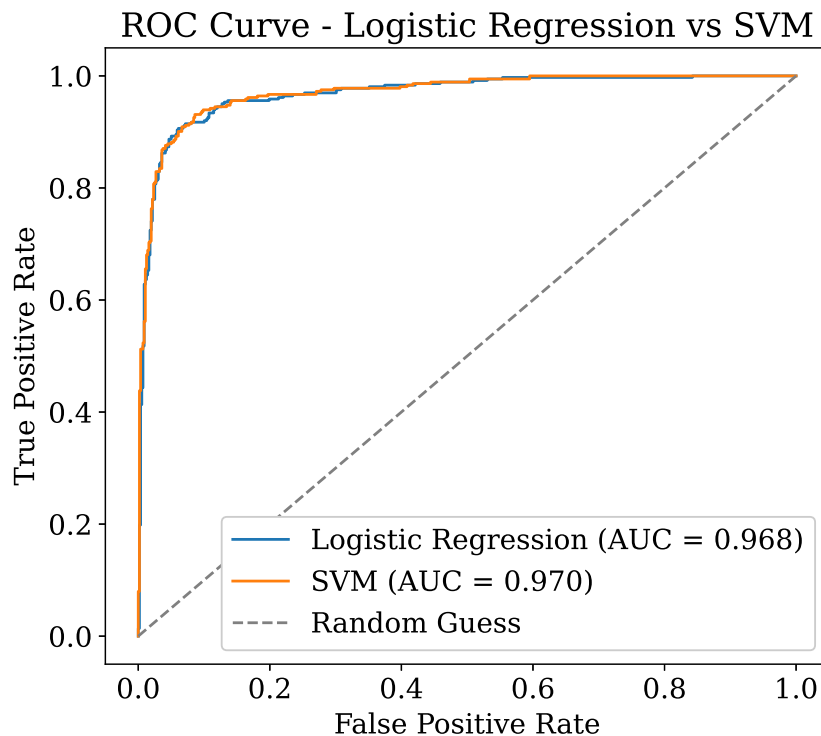


Figure 4: ROC curve comparison – Logistic Regression vs. SVM.

Inference: Both ROC curves rise steeply toward the top-left corner and lie well above the diagonal random-guess line, indicating strong discriminative ability for both models across decision thresholds. The curves are visually close, reflecting the similarity in AUC/accuracy already observed in the tabulated results, with neither model showing a decisive ranking advantage over the other.

11 Discussion

The tuned Logistic Regression model achieved the highest single-split test accuracy (0.9262) and F1-score (0.9259), narrowly outperforming the tuned SVM (accuracy 0.9207) on the held-out test set. However, under 5-fold cross-validation, SVM achieved a higher average accuracy (0.9356 vs. 0.9261), suggesting the RBF kernel generalizes marginally better across different data splits by capturing non-linear structure in the feature space, at the cost of higher training time. Regularization strength strongly influenced Logistic Regression: the best model chose $C = 100$ (weak regularization) with L1 penalty, indicating the full feature set (with implicit sparsity) was preferable to a heavily regularized, simpler model. Among SVM kernels, Linear and RBF performed best, while Polynomial underperformed badly with default settings – confirming that kernel choice and hyperparameter tuning are critical for SVM, more so than for the comparatively robust Logistic Regression.

12 Conclusion

Logistic Regression and SVM classifiers were implemented and tuned on the Spambase dataset. The tuned Logistic Regression model achieved the best single-split test accuracy,

while the RBF-kernel SVM achieved the best average cross-validation accuracy, indicating both approaches are competitive on this dataset, with Logistic Regression offering greater interpretability and speed, and SVM (RBF) offering slightly better cross-validated generalization.

13 References

- [1] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] Scikit-learn Developers, “Logistic Regression,” https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression.
- [3] Scikit-learn Developers, “Support Vector Machines,” <https://scikit-learn.org/stable/modules/svm.html>.
- [4] Spambase Dataset, Kaggle, <https://www.kaggle.com/datasets/somesh24/spambase>.